11.6.1 Selecting the right cache node type

So far, you used the cache node type `cache.t2.micro`, which comes with one vCPU, ~0.6 GB memory, and low to moderate network performance. You used this node type because it's part of the Free Tier. But you can also use more powerful node types on AWS. The upper end is the `cache.r6gd.16xlarge` with 64 vCPUs, ~419 GB memory, and 25 Gb network.

As a rule of thumb: for production traffic, select a cache node type with at least 2 vCPUs for real concurrency, enough memory to hold your data set with some space to grow (say, 20%; this also avoids memory fragmentation), and at least high network performance. The `cache.r6g.large` is an excellent choice for a small node size: 2 vCPUs, ~13 GB, and up to 10 GB of network. This may be a good starting point when considering how many shards you may want in a clustered topology, and if you need more memory, move up a node type. You can find the available node types at https://aws.amazon.com/elasticache/pricing/.

11.6.2 Selecting the right deployment option

By replicating data, you can distribute read traffic to multiple nodes within the same replica group. By replicating data from the primary node to one or multiple replication nodes, you are increasing the number of nodes that are able to process read requests and, therefore, increase the maximum throughput. On top of that, by sharding data, you split the data into multiple buckets. Each bucket contains a subset of the data. Each shard is stored on a primary node and is optionally synchronized to replication nodes as well. This further increases the number of nodes available to answer incoming reads—and even writes.

Redis supports the concept of replication, where one node in a node group is the primary node accepting read and write traffic, while the replica nodes only accept read traffic, which allows you to scale the read capacity. The Redis client has to be aware of the cluster topology to select the right node for a given command. Keep in mind that the replicas are synchronized asynchronously. This means that the replication node eventually reaches the state of the primary node.

Both Memcached and Redis support the concept of sharding. With sharding, a single cache cluster node is no longer responsible for all the keys. Instead the key space is divided across multiple nodes. Both Redis and Memcached clients implement a